CURRICULUM & TECHNICAL ROADMAP

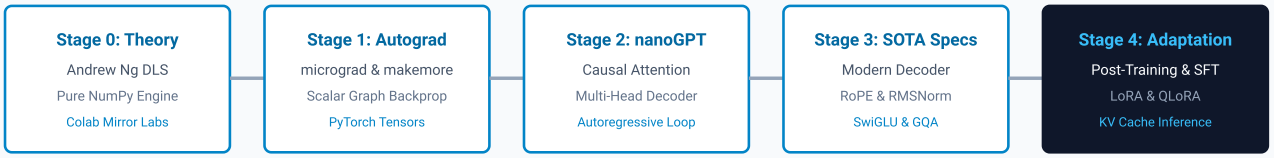
Deep Learning to Modern LLM Architecture

A First-Principles Strategy Bridging Classical Math with Code-First Transformer Engineering

Strategic Overview & Learning Philosophy

This roadmap bridges foundational calculus and neural mechanics with modern large language model engineering. By completing Andrew Ng's classical assignments in Google Colab and progressing through first-principles autograd and causal Transformer architectures in pure PyTorch, you develop production-grade intuition for modern Generative AI systems.

CURRICULUM PROGRESSION ARCHITECTURE



Stage 0 Execution: Running Course Assignments for Free

OPEN SOURCE PROTOCOL

Auditing the Coursera lectures yields core intuition, but coding the assignments cements vector mechanics. Because graded sandboxes require payment, use verified open-source GitHub mirrors executed in Google Colab.

Repository Mirror Workflow

- Locate GitHub repositories: `amanchadha/coursera-deep-learning-specialization` or `abdur75648/Deep-Learning-Specialization-Coursera`.
- Download the raw `.ipynb` assignment notebooks for all 5 courses.
- Import directly into a free Google Colab runtime.

Deliberate Practice Rule

- Clear all cell outputs before writing code.
- Independently complete blocks between `### START CODE HERE ###` and `### END CODE HERE ###`.
- Only inspect repository solutions after resolving matrix dimension errors.

Deep Learning Specialization: Critical Focus Areas for LLM Engineers

Course Name	Key Concepts to Master	Direct Application in LLMs
C1: Neural Networks & Deep Learning	Vectorization, NumPy matrix rank, forward/backprop, activation derivatives.	Calculating gradients in dense projection matrices (W_q, W_k, W_v, W_o).
C2: Improving Deep Neural Networks	Adam optimizer, weight initialization (Xavier/He), L2 decay, Dropout, Batch Norm.	Pretraining stability via AdamW, weight scaling, and learning rate warmups.
C3: Structuring ML Projects	Train/Dev/Test distributions, error analysis, orthogonalization, metric selection.	Diagnosing validation perplexity spikes and benchmark dataset contamination.
C4: Convolutional Neural Networks	Convolutions, stride, receptive fields, residual connections (ResNet).	Residual skip connections ($x + \text{SubLayer}(x)$) in Transformer blocks.
C5: Sequence Models	RNNs, LSTMs, GRUs, Word2Vec embeddings, Beam Search, Attention mechanism.	Token embeddings and self-attention mechanics prior to full Transformer stacks.

Stage 1: PyTorch Autograd & Neural Mechanics from Scratch

WEEKS 1–3

Transition from guided NumPy worksheets to bare-metal PyTorch. High-level frameworks hide computational graphs; implementing scalar auto-differentiation reveals how backpropagation operates dynamically across tensors.

1. Build micrograd (Scalar Autograd)

Milestone 1

Implement an autograd engine supporting addition, multiplication, and tanh/ReLU in pure Python:

- Construct directed acyclic graphs (DAG) tracking parent nodes.
- Implement backward closures calculating local derivatives.
- Execute topological sorting to call `.backward()` through the entire computational tree.

2. makemore Series (MLP to Tensors)

Milestone 2

Build character-level language models without high-level `torch.nn` containers:

- Transition from bigram frequency counts to Bengio 2003 MLP.
- Diagnose vanishing gradients and dead activations using tensor histograms.
- Hand-code backward passes for Batch Normalization and Cross-Entropy loss.

Stage 2: Causal Transformers & Pretraining from Scratch

WEEKS 4–7

Modern LLMs are autoregressive decoder-only Transformers. Construct every tensor transformation manually to understand how tokens attend to past context.

Component	Mathematical Operation & Shape Transform	Code Implementation Target
Scaled Dot-Product Attention	$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$ where M is lower-triangular causal mask.	Register causal buffer `torch.tril`; scale by $1/\sqrt{d_k}$; apply masked fill of $-\infty$.
Multi-Head Attention (MHA)	Linear projections split into H heads: $[B, T, C] \rightarrow [B, H, T, d_k]$. Concat heads and project via W_o.	Tensor transpose: `.view(B, T, H, d_k).transpose(1, 2)` for parallel matrix multiplies.
Pre-LayerNorm & Residuals	$x_{l+1} = x_l + \text{MLP}(\text{LN}(x_l + \text{MHA}(\text{LN}(x_l))))$. Stabilizes deep gradient backpropagation.	Apply normalization prior to attention/FFN blocks rather than after (post-LN legacy).
Autoregressive Sampling Loop	$\hat{y}_t \sim \text{Categorical}(\text{softmax}(z_t / T))$. Sample next token; append to context tensor.	Implement temperature scaling and Top-K filtering to control generation diversity.

Stage 2 Core Milestone: nanoGPT Reproduction

Follow Andrej Karpathy's *"Let's build GPT: from scratch"* and *"Let's reproduce GPT-2 (124M)"*. Write a single-file PyTorch training pipeline on TinyShakespeare or OpenWebText. Implement training loop mechanics including learning rate cosine annealing, gradient accumulation, and mixed-precision `torch.autocast(device_type='cuda', dtype=torch.bfloat16)`.

Stage 3: Modern Open-Weights LLM Architectures

WEEKS 8–9

GPT-2 established the baseline, but state-of-the-art open-weights architectures (Llama 3, Mistral, Gemma) incorporate four fundamental upgrades that you must implement in code.

1. Byte-Pair Encoding (minbpe)

Train custom BPE tokenizers from scratch. Learn how merge tables compress raw bytes into vocabulary tokens and why tokenization artifacts cause arithmetic and whitespace edge-case errors.

2. RMSNorm (Root Mean Square Norm)

Replace LayerNorm with RMSNorm: $\bar{a}_i = \frac{a_i}{\text{RMS}(a)} g_i$. Eliminates mean centering computations, reducing forward pass latency by 10–15% without loss of training stability.

3. Rotary Positional Embeddings (RoPE)

Replace absolute learned position embeddings with 2D rotation matrices applied directly to query and key vectors. Encodes relative token distance naturally across arbitrary sequence lengths.

4. SwiGLU & Grouped-Query Attention

Implement SwiGLU feed-forward networks: $\text{SwiGLU}(x) = (xW \cdot \text{SiLU}(xV))W_2$. Pair with GQA to share Key/Value heads across Query heads, slashing inference VRAM footprint.

Stage 4: Post-Training, Fine-Tuning & Efficient Inference

WEEKS 10–12

Pretraining creates a base completion engine. Post-training aligns the model into an instruction-following assistant capable of specialized reasoning and efficient production deployment.

Parameter-Efficient Fine-Tuning (PEFT / LoRA) Mechanics

Fine-tuning 7B+ parameter matrices directly is computationally prohibitive. Low-Rank Adaptation (LoRA) freezes the base weight $W_0 \in \mathbb{R}^{d \times k}$ and injects trainable rank-decomposition matrices $\Delta W = B \cdot A$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $r \ll \min(d, k)$. QLoRA quantizes W_0 to 4-bit NormalFloat while computing gradients through FP16/BF16 adapters.

Master Milestone Checklist & Deliverables

Stage	Target Duration	Key Technical Skill Acquired	Tangible Portfolio Deliverable
Stage 0	3–4 Weeks	NumPy vectorization, matrix calculus, optimization.	5 DLS assignment suites completed in Colab.
Stage 1	2–3 Weeks	Autograd graphs, tensor backpropagation, diagnostics.	Custom `micrograd` engine & character MLP.
Stage 2	3–4 Weeks	Causal self-attention, MHA, autoregressive pretraining.	Trained GPT-2 reproduction from an empty file.
Stage 3	2 Weeks	BPE tokenization, RoPE, RMSNorm, SwiGLU, GQA.	Standalone Llama 3 decoder in raw PyTorch.
Stage 4	3 Weeks	Supervised fine-tuning, LoRA/QLoRA, KV caching.	Instruction-tuned 1B–3B model with optimized inference.